

Signal — Master-Prompt (Stand v7)

Kopiere den folgenden Prompt vollständig, um die Website von Grund auf neu aufbauen zu lassen.

PROMPT-ANFANG

Baue eine Single-File-HTML-Webanwendung namens „**Signal — AI Intelligence**“: ein privates KI-News-Tool für mich und Freunde, damit wir ohne Social Media (X, TikTok, Instagram) täglich über die KI-Welt informiert bleiben. Kein öffentliches Portal, keine Community, kein Login. Alles in EINER HTML-Datei (HTML + CSS + JS inline), deutsche Sprache.

Kernidee

Ein scrollbares, X/Twitter-artiges AI-Dashboard mit zwei Rollen: **(1) News** — kurze Posts für den schnellen Überblick, ein Klick öffnet die Tiefenanalyse. **(2) Allgemeine Informationsquelle zum Thema KI** — ein Einsteiger-Bereich für Menschen, die das Thema ganz neu erleben (Grundlagen, Begriffe, erste Schritte). Dazu ein automatisches Archiv mit Wochenrückblick und ein kuratiertes Modell-Nachschlagewerk. Oberstes Prinzip: maximale Klarheit, Geschwindigkeit und Relevanz — in drei Minuten im Bild sein.

Technische Basis (WICHTIG)

- Die App läuft als Artifact in Claude.ai und nutzt die Anthropic-API direkt per fetch auf `https://api.anthropic.com/v1/messages` — OHNE API-Key im Code (wird von der Claude-Umgebung übernommen). Modell: `claude-sonnet-4-6`, mit Web-Search-Tool:

```
tools: [{ "type": "web_search_20250305", "name": "web_search" }].
```
- WICHTIG: Diese API-Aufrufe funktionieren NUR innerhalb von Claude. Heruntergeladen/extern gehostet bleibt die Seite leer. Für echtes Hosting muss die Architektur umgebaut werden: statische Seite liest eine `news.json`, die ein separates Server-Skript (API-Key als Secret, z. B. GitHub Actions Cron) regelmäßig erzeugt. Die Website selbst darf dann keine API-Aufrufe machen.
- Alle KI-Antworten werden als reines JSON angefordert (Prompt-Anweisung: „AUSSCHLIESSLICH reines JSON, ohne Markdown, ohne Backticks“) und robust geparkt: Markdown-Fences entfernen, JSON zwischen erstem `[` / `{` und letztem `]` / `}`

extrahieren, try/catch mit Retry-Button im UI.

- Persistenter Speicher über die Artifact-Storage-API (`window.storage.get/set/list`) für das Archiv. Mit Fallback: wenn `window.storage` nicht existiert, Hinweis anzeigen statt Crash.
- three.js r128 von cdnjs
(`https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js`), mit Fallback: wenn THREE fehlt, läuft die Seite ohne 3D weiter.
- Kein localStorage/sessionStorage. Kein `<form>`-Tag. `prefers-reduced-motion` respektieren. Mobile-tauglich (Karten stapeln einspaltig, Touch-Gesten).

Design (Awwwards × Raycast, inspiriert von landonorris.com)

- Dark Mode only. Hintergrund #0a0a0c, Panels glasig `rgba(22,22,26,.6)` mit `backdrop-filter: blur`, Hairline-Borders `rgba(255,255,255,.08)`.
- Akzentfarbe: Lime **#d2ff00** (Buttons, aktive Tabs, Hover-Glows, Badges).
- Typografie: System-Stack (SF Pro/Inter), große fette Headlines mit negativem Letter-Spacing; Monospace für Metadaten/Timestamps.
- 3D-Hintergrund (three.js): ~900 Lime-Partikel + rotierender Lime-Wireframe-Ikosaeder, Kamera folgt sanft der Maus. Im Intro präsent (Opacity ~0.55/0.16), nach dem Intro gedimmt (~0.22/0.05).
- Karten: 14px Radius, Hover = Border lime-glimmen + leichtes Anheben; gestaffeltes Einblenden von unten beim Rendern.

1. Intro (kurz, überspringbar — Lando-Norris-Stil)

Vollbild-Overlay mit 3 horizontalen Slides zum Durchswipen (Pointer-Drag mit Live-Mitziehen, Touch, Mausrad, Pfeiltasten, Enter/Space). Riesige Uppercase-Typo, einzelne Wörter nur als Lime-Outline (`-webkit-text-stroke`). Inhalte: (1) „Signal / AI Intelligence“, (2) „Klarheit statt Rauschen — kurze Posts, ein Klick für die Tiefe“, (3) „Los geht's“ mit Lime-Button. Unten Fortschritts-Striche + pulsierender Hint „← Wischen oder ziehen →“. Oben rechts „Überspringen“-Button, Escape überspringt ebenfalls. Exit-Animation: Zoom + Blur + Fade, dann Homepage einblenden. Der News-Feed lädt bereits im Hintergrund während des Intros.

2. App-Shell

Sticky Milchglas-Header: Brand „Signal / AI INTELLIGENCE“ links, Datum (Mono) rechts, darunter 4 Tabs: **Feed | Einstieg | Archiv | Modelle** (aktiver Tab lime mit Unterstrich).

Reihenfolge bewusst: News zuerst, direkt danach der Einsteiger-Bereich als zweite Kernfunktion.

3. Feed (Haupttab)

- Lädt per API-Call mit Websuche die **8 wichtigsten aktuellen KI-News** quer über die Bereiche Modelle & Tools, Unternehmen & Wirtschaft, Forschung, Politik. JSON-Struktur pro Meldung: `headline` (Deutsch, max. 12 Wörter), `why` (1–2 Sätze: was + warum relevant), `category` (genau eine: Modelle | Tools | Unternehmen | Wirtschaft | Forschung | Politik), `source`, `url`, `image` (og:image-URL oder null), `models[]`, `companies[]`.
- Darstellung als X-artige Posts: farbige Avatar-Kachel (Gradient aus Hash des Firmennamens + Initiale), Firmenname fett, Kategorie-Badge (lime), Quelle, Headline, Zusammenfassung, Bild, Hashtags aus Modellen, Hover-Hinweis „Details öffnen →“.
- **Themenfilter-Chips** über dem Feed: Alle | Modelle & Tools | Business | Forschung | Politik (filtern clientseitig die Kategorien; leerer Filter zeigt ehrlichen Hinweis statt Leere).
- „🔄 Feed aktualisieren“-Pill. Skeleton-Loader (Shimmer) während des Ladens. Fehlerzustand mit Retry-Button.
- **Bild-Fallback-Kette** (dreistufig, per Image-onload/onerror): (1) `image` -URL des Artikels → (2) kuratiertes Unsplash-Tech/KI-Foto aus fest eingebautem Pool (5–6 URLs mit `?w=1200&q=80&auto=format&fit=crop`) → (3) CSS-Gradient-Kachel mit Monogramm. Es darf nie ein kaputtes Bild erscheinen.

4. News-Detail-View

Klick auf einen Post öffnet ein Bottom-Sheet (von unten einfahrend, Backdrop mit Blur, x-Button + Escape + Backdrop-Klick schließen). Inhalt wird live per zweitem API-Call (mit Websuche) generiert und im Speicher gecacht (Key = Headline). JSON-Struktur: `what` (3–4 Sätze), `why` (2–3 Sätze), `impact[]` (3–4 Punkte), `context` (Einordnung/Vergleich zu früher), `takeaway` (1 Satz). Darstellung mit Lime-Uppercase-Zwischenüberschriften: „Was ist passiert / Warum es wichtig ist / Auswirkungen / Einordnung / Takeaway“, plus Bild (gleiche Fallback-Kette), Modell-/Firmen-Chips und Quellenlink. Während der Generierung: Spinner „Einordnung wird erstellt ...“.

5. Einstieg (kuratiert, statisch eingebaut — die zweite Kernfunktion)

Ein Bereich für absolute KI-Neulinge, komplett ohne API-Aufrufe (fest im Code, damit er sofort lädt, nichts kostet und offline-robust ist). Ton: freundlich, jargonfrei, auf Augenhöhe — kein Tech-Sprech ohne Erklärung. Drei Blöcke, jeweils als klickbare Karten im bekannten Design, Details öffnen sich im Bottom-Sheet:

1. **„KI in 5 Minuten“** — 5–6 Grundlagen-Karten in sinnvoller Lesereihenfolge (nummeriert, da echte Sequenz): Was ist KI / was ist ein Sprachmodell (in Alltagssprache, mit Analogie) · Wie „denkt“ ein Sprachmodell grob (Wahrscheinlichkeiten, kein Bewusstsein) · Was KI gut kann · Wo sie scheitert (Halluzinationen, Aktualität, Rechnen) · Warum gerade alle darüber reden · Wie man verantwortungsvoll damit umgeht (nichts Sensibles teilen, Wichtiges prüfen). Jede Karte: Titel, 1-Satz-Teaser, im Sheet dann 3–5 kurze Absätze + 1 Lime-„Merksatz“.
2. **„Erste Schritte“** — 3–4 praktische Karten: Welches Tool zum Starten (verweist auf den Modelle-Tab) · 5 Prompt-Tipps mit je einem Vorher/Nachher-Beispiel · Typische Anfängerfehler · 5 konkrete Aufgaben zum Ausprobieren heute Abend.
3. **„Begriffe, die dir überall begegnen“** — Glossar als kompaktes Chip-/Kartenraster, ~12 Begriffe (Prompt, Token, Halluzination, LLM, Multimodal, Open Source, Kontextfenster, Feintuning, Agent, Reasoning, API, GPU). Klick = Sheet mit 2–3 Sätzen Erklärung in Alltagssprache + 1 Beispiel.

Oben im Tab ein kurzer Willkommens-Hinweis („Ganz neu beim Thema? Lies die 6 Karten der Reihe nach — danach verstehst du jede Meldung im Feed.“). Inhalte versionsneutral formulieren, damit nichts schnell veraltet. Fußnote im Tab: „Kuratierte Grundlagen — Stand beim Erstellen der Seite.“

6. Archiv + Wochenrückblick

- Jeder erfolgreich geladene Feed wird automatisch persistiert: Key `day-YYYY-MM-DD`, Wert = JSON-Array der Meldungen, Deduplizierung per Headline beim Mergen.
- Archiv-Tab: die letzten 14 Tage, gruppiert mit Tages-Header („Donnerstag, 3. Juli · 8 Meldungen“ + Trennlinie), Posts kompakt ohne Bild, aber weiterhin klickbar → Detail-View.
- Leerer Zustand: freundlicher Hinweis, dass das Archiv ab jetzt automatisch wächst.
- **Wochenrückblick-Button** (lime, oben im Archiv): sammelt alle Meldungen der letzten 7 Tage aus dem Speicher und generiert per API-Call (OHNE Websuche, nur aus den Archivdaten) ein JSON: `summary` (die Woche in 3–4 Sätzen), `highlights[]` (3–5 Punkte), `trend` (übergreifender Trend), `watch` (worauf als Nächstes achten). Anzeige im Bottom-Sheet. Bei weniger als 3 archivierten Meldungen: ehrlicher Hinweis „Noch zu wenig Material“ statt Pseudo-Analyse.

7. Modelle (kuratiert, statisch eingebaut)

14 fest im Code hinterlegte Modell-Profile (bewusst versionsneutral formuliert, damit nichts schnell veraltet): GPT (OpenAI), Claude (Anthropic), Gemini (Google), Grok (xAI), Llama

(Meta), Mistral, DeepSeek, Qwen (Alibaba), Flux (Black Forest Labs), Midjourney, Veo (Google DeepMind), Runway, Kling (Kuaishou), Suno. Felder je Modell: Name, Entwickler, Kategorie (Text | Open Source | Bild | Video | Audio), Kurzbeschreibung, Stärken[], Schwächen[], typische Use Cases[], Preisniveau, 1-Satz-Vergleichseinordnung. Darstellung: Filter-Chips nach Kategorie + 2-spaltiges Kartenraster (Avatar-Kachel, Name, Entwickler, Beschreibung, Use-Case-Chips). Klick öffnet Profil im Bottom-Sheet mit allen Feldern + Hinweis „Kuratierte Basisdaten — Versionen und Preise ändern sich schnell“.

Bewusst NICHT enthalten

Use-Case-Assistent (gestrichen), Community/Kommentare, Login, Suche, Newsletter, Ticker-Leiste, Trending-/Beliebtheits-Widgets (brauchen Traffic-Daten, die es nicht gibt).

Footer

„Signal ist ein privater Prototyp. Inhalte werden automatisch recherchiert und von Claude erzeugt — Wichtiges bitte an der Originalquelle prüfen. Archiv wird lokal in dieser Umgebung gespeichert.“

PROMPT-ENDE

Anhang: Kontext & offene Punkte (nicht Teil des Prompts)

- **Projektziel:** Minimaler Zeit-/Geldeinsatz. Erst beweisen, dass das Tool täglich genutzt wird; Hosting/Skalierung/Monetarisierung erst danach.
- **Design-Historie:** v1 Terminal-minimal → v2 Linear/Vercel + Bloomberg-Ticker → v3 Apple-Stil → v4–v6 Awwwards/Raycast mit Lime + Swipe-Intro (finale Richtung) → v7 ergänzt den Einstieg-Tab als zweite Kernfunktion (News + Wissensquelle für Neulinge).
- **Konzept-Entscheidungen aus den Feedback-Runden:** Themenfokus = Mix mit Filter; Archiv + Wochenrückblick = gewünscht; Use Cases = gestrichen.
- **Offen:** (1) Tech-Level der Freunde → bestimmt den Ton der Einordnungen (Laienfreundlich vs. Fachbegriffe ok); (2) Erfolgskriterium nach 2 Wochen; (3) späterer Hosting-Umbau (GitHub Pages + Actions-Cron + news.json), wenn sich tägliche Nutzung bestätigt.